DuocUC | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

# ATLAS

## Plan de pruebas de seguridad y calidad en el desarrollo de software

**Ramo ISY1102 - Seguridad y calidad en el desarrollo de software**

Sección 001V


**Integrantes**

Mario Cofré

Nelson Cofré

Nicolás Sánchez


**Carrera**

Ingeniería en informática (Desarrollo de software)

**Fecha**

30-08-2026

**Docente**

Juan Guillermo Azocar Malverde

## 1. Introducción

Atlas es una plataforma web multicapa desarrollada por CreaLab SpA orientada a la gestión centralizada de clientes, contratos y documentos digitales para pequeñas y medianas empresas (pymes) en Chile.

Debido a que este almacena y procesa información comercial, contractual y datos personales sensibles (como RUT e información bancaria de clientes)], el desarrollo exige una verificación técnica rigurosa que garantice la estabilidad operativa y la protección contra accesos no autorizados.

La verificación técnica de Atlas será hecha en base a sus principales componente:

- Frontend: Una aplicación de página única (SPA) desarrollada en React (utilizando Material UI, Tailwind CSS y Axios). Incluye requisitos específicos de accesibilidad y usabilidad orientados a usuarios adultos mayores (controles de interfaz ampliados, tipografía de al menos 12px, alto contraste y etiquetas ARIA).
- Backend: Una API RESTful implementada en Node.js con Express que gestiona la lógica de negocio y las operaciones sobre los datos.
- Base de Datos y Almacenamiento: Base de datos relacional PostgreSQL para la persistencia de empresas, usuarios, clientes y contratos, junto con un sistema de almacenamiento interno para archivos adjuntos (PDF, DOC, etc.).
- Seguridad Integrada: Comunicaciones cifradas obligatorias mediante TLS, contraseñas protegidas con algoritmos de hashing bcrypt, gestión de sesiones a través de tokens JWT (access y refresh token) y control de acceso basado en roles (RBAC con perfiles de Administrador y Editor/Visualizador).

El objetivo de este plan de pruebas desarrollado bajo el estándar “IEEE 29119”, es establecer una estrategia integrada de aseguramiento de calidad y seguridad desde las etapas tempranas del desarrollo.

El plan busca garantizar el cumplimiento simultáneo de los **tres pilares de la confianza digital**

1) Calidad de Software (QA): Evaluar la funcionalidad de los flujos críticos (autenticación, gestión de empresas, clientes y contratos) según el modelo “ISO/IEC 25010”, verificando tiempos de respuesta (< 300 ms en operaciones CRUD simples y < 2 s en la carga inicial) y la usabilidad adaptada a adultos mayore
2) **Seguridad en el Ciclo de Vida (DevSecOps):** Preservar la “tríada CIA (Confidencialidad, Integridad y Disponibilidad)” mediante la aplicación de análisis estático (**SAST**), dinámico

(**DAST**) y de dependencias (**SCA**), previniendo vulnerabilidades del **OWASP Top 10** como Inyección SQL, Cross-Site Scripting (XSS) y Cross-Site Request Forgery (CSRF).

3) **Cumplimiento Normativo y Gobernanza (Compliance):** Asegurar la conformidad con el marco legal chileno, destacando la **Ley N° 21.663 (Ley Marco de Ciberseguridad en Chile)** y las normativas de protección de datos personales, garantizando además la generación de **registros de auditoría (Audit Trails)** inmutables para cada acción crítica (creación, edición, eliminación o descarga de documentos).

Para ello se evaluaran los siguientes puntos en el plan de pruebas

**1) Establecer los Criterios de Calidad, Seguridad y Cumplimiento Normativo:** Definición de métricas de calidad (ISO/IEC 25010), riesgos de seguridad (OWASP) y regulaciones legales (Ley N° 21.663).

**2) Definir los Tipos de Pruebas y su Justificación:** Clasificación de pruebas funcionales (unitarias, integración, sistema/E2E, UAT) y no funcionales (rendimiento, usabilidad WCAG, compatibilidad, SAST/DAST).

**3) Realizar una estrategia de Pruebas y Matriz de Riesgos:** Delimitación del alcance (**In-Scope vs. Out-Scope**) y análisis de riesgos técnicos y de negocio con sus planes de mitigación.

**4) Recursos Necesarios y Cronograma:** Detalle de perfiles humanos, entornos de prueba y herramientas técnicas (Jira, SonarQube, JMeter, OWASP ZAP) organizadas en un cronograma de ejecución.

**5) Diseño de Casos de Prueba:** Matriz con al menos 5 casos de prueba vinculados a los códigos de la Especificación de Requerimientos de Software (ej. RF-1.1, RF-3.2, RF-4.2, RF-5.2, NFR-USAB-1), detallando ID, descripción, datos de entrada, resultado esperado, tipo de prueba y criterios de aceptación.

## 2. Criterios de calidad, seguridad y cumplimiento normativo a satisfacer en el caso

### 2.1. Calidad (Usabilidad, Rendimiento, Estabilidad, Confiabilidad y Mantenibilidad)

- **Usabilidad:**
    - **Accesibilidad e Inclusión (Adultos Mayores - *NFR-USAB-1*):** Interfaz desarrollada con tipografía legible de tamaño base mínimo de 12px, esquemas de alto contraste visual y etiquetas semánticas **ARIA** compatibles con lectores de pantalla.
    - **Diseño Responsivo (*NFR-COMPAT-2*):** Adaptabilidad visual y operativa en dispositivos de escritorio (*Desktop*), tabletas y teléfonos móviles.
    - **Facilidad de Navegación y Jerarquía (*NFR-USAB-2*, *NFR-USAB-4*):** Flujos simplificados con baja carga cognitiva, organizados secuencialmente (selección de empresa → cliente→ contratos del cliente).
    - **Criterio de Aceptación:** Validación mediante pruebas de usuario con 5 adultos mayores representativos sin registrar bloqueos mayores.
- **Rendimiento (*NFR-PERF-1*, *NFR-PERF-2*, *NFR-PERF-3*):**
    - **Latencia de Operaciones CRUD:** Tiempo de respuesta inferior a **300 ms** en consultas y persistencia simple en condiciones normales de carga.
    - **Tiempo de Carga Inicial:** Despliegue de la página principal en menos de **2.0 segundos** en redes móviles promedio.
    - **Gestión de Carga Concurrente:** Capacidad de respuesta estable bajo una carga estimada de al menos **200 usuarios concurrentes** simultáneos.
- **Estabilidad y Confiabilidad (*NFR-DIS-1*, *NFR-DIS-2*, *NFR-SEG-7*):**
    - **Disponibilidad Continua (*Uptime*):** Garantía de operatividad del sistema de al menos un **99.5% mensual**.
    - **Recuperación y Resiliencia:** Políticas de respaldo periódico (*backups*) automáticos y verificables en la base de datos PostgreSQL y el almacenamiento interno de archivos (*RF-5.3*), asegurando la continuidad operativa ante fallos.
    - **Ventanas de Mantenimiento:** Procedimientos de actualización y migración de esquemas de datos seguros y controlados.
- **Mantenibilidad:**
    - **Arquitectura Modular:** Separación desacoplada entre Frontend (*React SPA*) y Backend (*Express REST API*).
    - **Calidad del Código:** Código estructurado, reutilizable y sujeto a pruebas unitarias y de integración para facilitar futuras extensiones.

## 2.2. Seguridad (Protección de datos, Control de acceso, Manejo de errores e Integridad de la información)

- **Protección de Datos (*NFR-SEG-1*, *NFR-SEG-3*, *NFR-SEG-8*):**
    - **Cifrado en Tránsito:** Uso mandatorio y exclusivo del protocolo **TLS (HTTPS)** para todas las comunicaciones entre cliente, API y base de datos.
    - **Cifrado en Reposo:** Encriptación de datos sensibles de clientes (RUT, contactos y datos bancarios según *RF-4.1*).
    - **Protección Criptográfica de Contraseñas (*NFR-SEG-2*):** Almacenamiento irreversible mediante algoritmos de hashing **bcrypt** con sal (*salt*) aleatoria y factor de costo computacional alto.
- **Control de Acceso (*NFR-SEG-4*, *NFR-SEG-9*, *RB-2*, *RB-4*, *RB-5*):**
    - **Control de Acceso Basado en Roles (RBAC):** Restricción estricta donde el perfil **Administrador** tiene permisos totales (*CRUD* sobre clientes, contratos y usuarios), mientras que el rol **Editor / Visualizador** tiene acceso restringido exclusivamente a la lectura de datos (*RB-4*).
    - **Aislamiento Multinquilino (Multitenancy):** Validación en backend para que los usuarios solo puedan acceder a los registros vinculados a su empresa (empresa_usuarios), previniendo vulnerabilidades de acceso directo a objetos (IDOR / BOLA).
    - **Gestión de Sesiones Seguras:** Autenticación mediante tokens **JWT** de corta vigencia, combinados con mecanismos de revocación forzosa al ejecutar el cierre de sesión (*RF-1.2*).
- **Manejo de Errores (*NFR-USAB-3*, *NFR-SEG-5*):**
    - **Mensajes Amigables y Guiados:** En la interfaz de usuario, los errores de validación se presentan de forma clara y constructiva.
    - **Prevención de Exposición de Información (*Information Leakage*):** En el backend, supresión de trazas técnicas de la base de datos o stack traces en respuestas HTTP de error, devolviendo únicamente códigos de estado estándar y mensajes genéricos.
- **Integridad de la Información (*NFR-SEG-5*, *RF-4.3*, *RF-5.3*):**
    - **Saneamiento y Validación de Entradas:** Validación estricta *server-side* para prevenir vulnerabilidades de **Inyección SQL**, **Cross-Site Scripting (XSS)** y **CSRF**.
    - **Consistencia Transaccional y Borrado Lógico:** Cumplimiento ACID en PostgreSQL, solicitando confirmación de usuario y aplicando eliminación lógica (*soft delete*) para evitar pérdidas de datos accidentales (*RF-4.3*).

## 2.3. Normativas (Estándares técnicos, Leyes y Políticas internas)

- **Estándares Técnicos Internacionales:**
    - **ISO/IEC 25010:2011:** Marco rector para la evaluación de los atributos de calidad de software (usabilidad, rendimiento, confiabilidad, seguridad y compatibilidad).
    - **ISO/IEC 27001:2022:** Alineación con las directrices de un Sistema de Gestión de Seguridad de la Información (**SGSI**), contemplando gestión de accesos, cifrado y trazabilidad.
    - **OWASP Top 10:** Estándar de seguridad para la mitigación de las 10 vulnerabilidades más críticas en aplicaciones y APIs web.
    - **Pautas WCAG 2.1 (Nivel AA):** Estándar de accesibilidad web aplicado al diseño frontend.
    - **Buenas Prácticas PCI-DSS:** Directrices para el tratamiento seguro, enmascaramiento y no exposición en texto plano de datos de cuentas bancarias (*RF-4.1*).
- **Marco Legal (Leyes Chilenas Aplicables):**
    - **Ley N° 21.663 (Ley Marco de Ciberseguridad en Chile):**
        - Implementación del principio de **Seguridad por Diseño (*Security by Design*)** desde las etapas tempranas del ciclo de vida del software.
        - Preparación de mecanismos de monitoreo y registro para la notificación de incidentes de seguridad exigidos por la **Agencia Nacional de Ciberseguridad (ANCI)**.
    - **Ley N° 21.719 (Protección de Datos Personales en Chile):**
        - **Principio de Minimización de Datos:** Recolección estricta de información necesaria para la relación comercial y contractual.
        - **Garantía de Derechos ARCO:** Soporte técnico para la rectificación, actualización y supresión de datos personales del titular.
- **Políticas Internas y de Gobernanza (*RF-6.1*, *RB-1*, *NFR-SEG-6*):**
    - **Pistas de Auditoría Inmutables (*Audit Trails*):** Registro automático y obligatorio en base de datos de cada acción crítica (creación, edición, eliminación o descarga de clientes, contratos y usuarios), capturando ID de usuario, rol, IP de origen y marca de tiempo UTC (*RF-6.1*).
    - **Políticas de Contraseñas Corporativas:** Longitud mínima obligatoria de 12 caracteres combinando mayúsculas, minúsculas, números y símbolos.
    - **Unicidad de Registros Principales (*RB-1*):** Restricción de que un mismo correo no pueda existir como cuenta principal en dos empresas distintas si la política institucional así lo determina.

## 3. Plan de pruebas

### 3.1 Estrategia de pruebas

### Enfoque de pruebas

Se utilizará un enfoque mixto de pruebas manuales y automatizadas.

Las pruebas automatizadas serán aplicadas principalmente en funcionalidades críticas y repetitivas, como autenticación, validación de endpoints, control de acceso por roles y operaciones CRUD. Esto permitirá ejecutar las pruebas de manera frecuente y detectar posibles errores producidos por modificaciones en el código.

Por otra parte, las pruebas manuales serán utilizadas para evaluar aspectos que requieren interacción humana, especialmente la usabilidad, accesibilidad y experiencia de navegación. Estas pruebas son relevantes para Atlas debido a que el sistema considera entre sus usuarios a personas adultas mayores, por lo que se debe verificar que la interfaz sea clara, comprensible y fácil de utilizar.

### Etapas del proceso de pruebas

El proceso de pruebas se desarrollará mediante las siguientes etapas:

**1. Planificación de pruebas:**

Se identificarán los requisitos funcionales y no funcionales definidos en el ERS, estableciendo los objetivos, alcance, tipos de pruebas y criterios de aceptación.

**2. Diseño de casos de prueba:**

Se elaborarán casos de prueba asociados a los requerimientos del sistema, definiendo condiciones iniciales, datos de entrada, pasos de ejecución, resultados esperados y criterios de aceptación.

**3. Preparación del entorno de pruebas:**

Se configurará un ambiente controlado que permita ejecutar el sistema sin afectar datos reales. El entorno deberá incluir el frontend desarrollado en React, la API REST desarrollada con Express y una base de datos PostgreSQL.

**4. Ejecución de pruebas funcionales e integración:**

Se comprobará el correcto funcionamiento de los módulos principales del sistema, tales como autenticación, registro de usuarios, gestión de empresas, clientes, contratos, documentos y auditoría. Posteriormente se verificará la correcta comunicación entre los distintos componentes de la arquitectura.

**5. Ejecución de pruebas no funcionales:**

Se evaluarán requisitos relacionados con seguridad, rendimiento, carga, usabilidad, accesibilidad, compatibilidad y disponibilidad.

**6. Registro y corrección de incidencias:**

Los errores detectados serán documentados indicando su descripción, nivel de criticidad, módulo afectado y resultado esperado. Una vez corregidos, deberán ser nuevamente evaluados.

**7. Pruebas de regresión y cierre**:

Después de realizar modificaciones o correcciones, se ejecutarán nuevamente las pruebas necesarias para verificar que los cambios no hayan afectado funcionalidades que anteriormente funcionaban correctamente. El proceso finalizará cuando se cumplan los criterios de salida establecidos.

## Herramientas de apoyo

Para la ejecución de las pruebas podrán utilizarse herramientas especializadas de acuerdo con el tipo de prueba a realizar.

Para las pruebas de la API REST se puede utilizar Postman, permitiendo enviar solicitudes a los diferentes endpoints y validar respuestas, códigos HTTP y comportamiento de las funcionalidades.

Para pruebas automatizadas se pueden utilizar herramientas compatibles con JavaScript y Node.js, tales como Jest, para validar componentes o funcionalidades del sistema.

Las pruebas de seguridad pueden apoyarse en herramientas de análisis y escaneo de vulnerabilidades, mientras que las pruebas de rendimiento y carga pueden realizarse utilizando herramientas como Apache JMeter o k6.

Para verificar la compatibilidad y funcionamiento visual de la plataforma se utilizarán navegadores modernos como Google Chrome, Mozilla Firefox, Microsoft Edge y Safari, de acuerdo con los requisitos definidos para el sistema.

### Priorización de las pruebas

Las pruebas serán priorizadas considerando la criticidad de los requisitos y el impacto que podría generar una falla en el sistema.

En primer nivel de prioridad se encuentran las funcionalidades relacionadas con autenticación, gestión de sesiones, control de acceso por roles y protección de información, debido a que una falla en estos componentes podría permitir accesos no autorizados o comprometer información sensible.

También tendrán alta prioridad las operaciones de gestión de clientes, contratos y documentos, ya que constituyen las principales funcionalidades del sistema.

Posteriormente se evaluarán los requisitos relacionados con rendimiento, usabilidad, accesibilidad, compatibilidad y disponibilidad.

La priorización propuesta es la siguiente:

| **Prioridad** | **Componentes o funcionalidades** |
|---|---|
| Alta | Autenticación, JWT, control de acceso por roles, protección de datos y seguridad |
| Alta | Gestión de clientes, contratos y documentos |
| Alta | Auditoría de acciones críticas |
| Media | Rendimiento y carga |
| Media | Usabilidad y accesibilidad |
| Media | Compatibilidad entre navegadores y dispositivos |
| Alta | Aspectos visuales y mejoras no críticas de la interfaz |

### Criterios de entrada

Las pruebas podrán comenzar cuando se cumplan las siguientes condiciones:

- Los requerimientos a evaluar se encuentran definidos en el ERS.
- El ambiente de pruebas está correctamente configurado.
- Las funcionalidades principales se encuentren disponibles para ser evaluadas.
- Existen casos de prueba definidos para los requerimientos seleccionados.
- Se cuente con datos de prueba necesarios para la ejecución.
- Los componentes principales del sistema puedan comunicarse correctamente.

### Criterios de salida

El proceso de pruebas podrá considerarse finalizado cuando:

- Los casos de prueba planificados hayan sido ejecutados.
- Los requisitos funcionales críticos cumplan con sus criterios de aceptación.
- No existen vulnerabilidades críticas pendientes de resolución.
- Los requisitos de rendimiento definidos en el ERS hayan sido cumplidos.
- Se ha verificado el funcionamiento del sistema en los navegadores y dispositivos definidos.
- Los errores críticos y de alta prioridad han sido corregidos y validados.
- Las pruebas de regresión no presentan fallas críticas.

## 3.2 Tipos de prueba

Para asegurar la calidad del sistema Atlas se realizarán diferentes tipos de pruebas, seleccionadas de acuerdo con los requisitos funcionales y no funcionales definidos en la Especificación de Requerimientos de Software.

La combinación de estos tipos de pruebas permitirá evaluar tanto el comportamiento correcto de las funcionalidades como características de calidad relacionadas con seguridad, rendimiento, facilidad de uso, compatibilidad y estabilidad del sistema.

### 3.2.1 Pruebas funcionales

Las pruebas funcionales tienen como objetivo verificar que cada funcionalidad del sistema cumpla con el comportamiento especificado en los requisitos.

En Atlas, estas pruebas se aplicarán a los principales módulos definidos en el ERS:

- Autenticación e inicio de sesión.
- Cierre de sesión y revocación de tokens.
- Registro de usuarios.
- Creación y administración de empresas.
- Gestión de usuarios y asignación de roles.
- Gestión de clientes.
- Creación, edición y eliminación de contratos.
- Carga y descarga de documentos.
- Cálculo del estado de los contratos.
- Registro de acciones en el sistema de auditoría.

Estas pruebas son necesarias para verificar el cumplimiento de los requisitos funcionales RF-1 a RF-6, asegurando que las operaciones principales del sistema produzcan los resultados esperados.

También permitirán validar reglas de negocio, como la restricción de permisos de los usuarios con rol editor, quienes solo deben tener acceso de visualización, mientras que los administradores cuentan con permisos de creación, modificación y eliminación.

### 3.2.2 Pruebas no funcionales

Las pruebas no funcionales permiten evaluar características relacionadas con la calidad del sistema y no únicamente con las funcionalidades que este ofrece.

En el caso de Atlas, estas pruebas son especialmente importantes debido a que el sistema administra información de clientes, contratos y documentos, por lo que debe cumplir requisitos relacionados con seguridad, rendimiento, usabilidad, disponibilidad y compatibilidad.

Las principales pruebas no funcionales consideradas serán las siguientes:

- Pruebas de rendimiento y carga.
- Pruebas de seguridad.
- Pruebas de usabilidad y accesibilidad.
- Pruebas de compatibilidad.
- Pruebas de disponibilidad y recuperación.

La aplicación de estas pruebas permitirá verificar que el sistema no solo funcione correctamente, sino que también mantenga un nivel adecuado de calidad durante su utilización.

### 3.2.3 Pruebas de seguridad

Las pruebas de seguridad tienen como objetivo identificar posibles vulnerabilidades que puedan comprometer la confidencialidad, integridad y disponibilidad de la información.

Debido a que Atlas administra información de empresas, clientes, contratos y documentos, este tipo de prueba posee una alta prioridad dentro del plan.

Las pruebas deberán verificar:

- El correcto funcionamiento de la autenticación de usuarios.
- La protección de contraseñas mediante hashing seguro.
- El manejo adecuado de tokens JWT y sesiones.
- La correcta aplicación del control de acceso basado en roles.
- Que los usuarios no puedan acceder a información o funcionalidades para las cuales no poseen permisos.
- La validación y saneamiento de los datos ingresados por los usuarios.
- La protección frente a ataques como SQL Injection, Cross-Site Scripting (XSS) y Cross-Site Request Forgery (CSRF).
- El uso de comunicaciones seguras mediante TLS.
- La protección de datos sensibles almacenados.

- La generación segura de registros y auditorías.
- La disponibilidad de mecanismos de respaldo y recuperación.

Estas pruebas son necesarias para validar los requisitos NFR-SEG definidos en el ERS y reducir el riesgo de accesos no autorizados, manipulación de información o exposición de datos sensibles.

### 3.2.4 Pruebas de integración

Las pruebas de integración permiten comprobar que los diferentes componentes del sistema funcionen correctamente cuando interactúan entre sí.

Atlas posee una arquitectura multicapa compuesta principalmente por un frontend desarrollado en React, una API REST desarrollada en Express y una base de datos PostgreSQL. Por esta razón, no es suficiente probar cada componente de manera aislada.

Las pruebas de integración deberán verificar, entre otros aspectos:

- Comunicación entre el frontend y la API REST.
- Comunicación entre la API y la base de datos.
- Funcionamiento del proceso de autenticación mediante JWT.
- Acceso a endpoints protegidos según el rol del usuario.
- Creación de clientes y su almacenamiento en la base de datos.
- Asociación entre clientes y contratos.
- Carga de documentos y almacenamiento de su referencia.
- Registro de acciones críticas en el módulo de auditoría.

Estas pruebas son necesarias para detectar errores relacionados con la comunicación entre componentes, formatos incorrectos de datos, problemas de autenticación o fallas en el flujo completo de una funcionalidad.

### 3.2.5 Pruebas de usabilidad y accesibilidad

Las pruebas de usabilidad tienen como objetivo evaluar la facilidad con la que los usuarios pueden utilizar el sistema y completar las tareas principales.

En Atlas, estas pruebas son particularmente relevantes debido a que el sistema establece una consideración especial para usuarios adultos mayores.

Se evaluarán aspectos como:

- Claridad de los formularios.
- Facilidad para realizar el proceso de registro e inicio de sesión.
- Legibilidad de textos.
- Tamaño adecuado de botones y controles.
- Contraste entre elementos visuales.
- Claridad de los mensajes de error.
- Facilidad de navegación entre clientes, contratos y documentos.
- Navegación mediante teclado.
- Uso de etiquetas y atributos de accesibilidad.

Además, se podrán realizar pruebas con usuarios representativos para identificar dificultades durante la utilización de las principales funcionalidades.

Estas pruebas son necesarias para asegurar que la plataforma pueda ser utilizada de manera eficiente y comprensible por los diferentes tipos de usuarios definidos para el sistema.

### 3.2.6 Pruebas de rendimiento y carga

Las pruebas de rendimiento permiten medir la capacidad del sistema para responder dentro de los tiempos establecidos en los requisitos.

Para Atlas se deberán verificar los siguientes aspectos:

- Las operaciones CRUD simples deben presentar un tiempo de respuesta inferior a 300 milisegundos bajo condiciones normales.
- La página principal debe cargar en menos de 2 segundos en una conexión móvil promedio.
- El sistema debe ser capaz de soportar aproximadamente 200 usuarios concurrentes.
- La carga y descarga de documentos debe realizarse en un tiempo no superior a 10 segundos.

Las pruebas de carga permitirán evaluar el comportamiento del sistema cuando múltiples usuarios realizan solicitudes simultáneamente.

Estas pruebas son necesarias para identificar posibles problemas de lentitud, saturación de recursos o fallas cuando el sistema sea utilizado por varios usuarios al mismo tiempo.

Algunas herramientas que utilizaremos en este proceso son:

- Apache JMeter / k6: Simulan los 200 usuarios concurrentes bajo escenarios de carga y estrés para medir la estabilidad de la API y el pool de conexiones de PostgreSQL.

- Postman: Mide la latencia individual y tiempos de respuesta directos en los endpoints REST para verificar que las operaciones CRUD respondan en menos de 300 ms.
- Google Chrome DevTools (o Lighthouse): Evalúa el rendimiento de la carga inicial de la interfaz frontend (React SPA) para asegurar un despliegue en menos de 2 segundos en conexión móvil.

Más adelante habrá más ejemplos.

### 3.2.7 Pruebas de compatibilidad

Las pruebas de compatibilidad tienen como objetivo verificar que Atlas funcione correctamente en los diferentes navegadores y dispositivos definidos en los requisitos.

Se deberá comprobar el funcionamiento en las últimas versiones compatibles de:

- Google Chrome.
- Mozilla Firefox.
- Microsoft Edge.
- Safari.

Además, se verificará el comportamiento responsive de la interfaz en:

- Computadores de escritorio.
- Tablets.
- Dispositivos móviles.

Estas pruebas son necesarias debido a que Atlas es una plataforma web que debe ser accesible desde diferentes dispositivos y tamaños de pantalla.

### 3.2.8 Pruebas de disponibilidad y recuperación

*Las pruebas de disponibilidad y recuperación permiten verificar la capacidad del sistema para mantener la continuidad de sus servicios y recuperarse frente a posibles fallas.*

*Para Atlas se considerará el objetivo de disponibilidad mensual definido en el ERS, correspondiente a un 99,5%.*

*También se evaluarán los procedimientos relacionados con:*

- *Respaldo de información.*
- *Recuperación de datos.*
- *Restauración de servicios ante fallos.*

- *Actualizaciones del sistema.*
- *Migración segura de datos.*

*Estas pruebas son necesarias debido a la importancia de la información almacenada en el sistema, especialmente los datos de clientes, contratos y documentos asociados.*

### 3.2.9 Pruebas de regresión

Las pruebas de regresión serán utilizadas después de realizar modificaciones o correcciones en el sistema.

Su objetivo será comprobar que los cambios realizados no hayan generado errores en funcionalidades que anteriormente funcionaban correctamente.

Estas pruebas son especialmente importantes en funcionalidades críticas como autenticación, gestión de roles, clientes, contratos y documentos, debido a que una modificación en un componente puede afectar otras partes del sistema.

La aplicación de pruebas de regresión permitirá mantener la estabilidad y calidad de Atlas durante futuras etapas de desarrollo, corrección y mantenimiento.

### 3.3 Criterios de aceptación

Los criterios de aceptación permiten determinar de manera transversal si una prueba se considera aprobada o rechazada. Para el sistema Atlas, estos criterios se establecen considerando los requisitos funcionales, no funcionales y las reglas de negocio definidas en el ERS.

Una prueba será considerada aprobada cuando el resultado obtenido corresponda al resultado esperado y cumpla con los criterios establecidos para el requerimiento asociado. En caso contrario, la prueba será considerada fallida y deberá registrarse la incidencia correspondiente para su corrección y posterior ejecución de una prueba de regresión.

Los principales criterios de aceptación son los siguientes:

| Categoría | Criterio de aceptación |
|---|---|
| Funcionalidad | Las funcionalidades asociadas a los requisitos funcionales deben ejecutarse correctamente, sin errores que impidan completar el flujo correspondiente. |
| Autenticación y autorización | El sistema debe permitir el acceso únicamente a usuarios autenticados y respetar los permisos definidos según el rol del usuario. |
| Reglas de negocio | Se deben respetar las reglas RB-1 a RB-5, incluyendo las restricciones de acceso a empresas, asociación de contratos y permisos de los roles. |
| Documentos | La carga y descarga de documentos debe funcionar correctamente y realizarse en un tiempo no mayor a 10 segundos. El estado del contrato debe calcularse correctamente según sus fechas de inicio y término. |
| Auditoría | Las acciones críticas, como creación, edición, eliminación de clientes, contratos y usuarios, además de la subida y descarga de documentos, deben generar registros de auditoría. |
| Seguridad | Las pruebas de seguridad deben verificar los mecanismos de autenticación, autorización, protección de datos y validación de entradas. No deben existir hallazgos críticos en la auditoría de seguridad. |
| Rendimiento | Las operaciones CRUD simples deben |

| | |
|---|---|
| | presentar un tiempo de respuesta inferior a 300 ms bajo carga normal. La página principal debe cargar en menos de 2 segundos en una conexión móvil promedio. |
| Concurrencia | El sistema debe soportar la cantidad de usuarios concurrentes definida, establecida por defecto en 200 usuarios concurrentes. |
| Usabilidad y accesibilidad | Las interfaces deben presentar textos legibles, contraste adecuado, controles apropiados y flujos claros para las principales operaciones del sistema. |
| Disponibilidad y mantenimiento | Se debe considerar una disponibilidad objetivo de 99,5% mensual y que las actualizaciones y migraciones de datos se realicen de forma segura. |
| Compatibilidad | La aplicación debe funcionar correctamente en las últimas dos versiones de Chrome, Firefox, Safari y Edge, además de adaptarse a dispositivos móviles y tablets. |
| Defectos | No deben quedar defectos críticos o de alta prioridad abiertos al finalizar la ejecución de las pruebas. Los defectos de menor prioridad deben quedar registrados y documentados para su seguimiento. |

Como criterio general de salida, se considerará que el plan de pruebas ha cumplido sus objetivos cuando los casos de prueba de prioridad alta hayan sido ejecutados, los requisitos críticos hayan sido validados, los defectos críticos hayan sido corregidos y las pruebas de regresión hayan confirmado que las correcciones no afectan funcionalidades previamente aprobadas.

### 3.4 Herramientas utilizadas

| Herramienta | Uso en las pruebas | Fundamento |
|---|---|---|
| Postman | Pruebas funcionales y de integración de la API REST. | Permite enviar solicitudes HTTP y verificar respuestas, códigos de estado, datos y comportamiento de los endpoints. |

| | | |
|---|---|---|
| Jest | Pruebas automatizadas. | Permite automatizar la validación de componentes y funciones del sistema, facilitando la detección temprana de errores. |
| Supertest | Pruebas de endpoints de la API. | Permite comprobar directamente el comportamiento de la API Express y validar las respuestas entregadas por sus endpoints. |
| OWASP ZAP | Pruebas de seguridad. | Permite analizar la aplicación en busca de vulnerabilidades relacionadas con validación de entradas, XSS, configuraciones inseguras y otros riesgos. |
| JMeter / k6 | Pruebas de rendimiento y carga. | Permite simular múltiples usuarios y solicitudes concurrentes para comprobar los tiempos de respuesta y el comportamiento del sistema bajo carga. |
| Lighthouse | Pruebas de rendimiento, accesibilidad y calidad del frontend. | Permite obtener métricas sobre carga, accesibilidad y buenas prácticas de la aplicación web. |
| Entrar con distintos navegadores.<br>EJM: Opera GX | Pruebas de compatibilidad y comportamiento responsive. | Permiten revisar el comportamiento de la aplicación en diferentes resoluciones, dispositivos simulados y navegadores. |
| Herramientas de desarrollador del navegador | Pruebas de compatibilidad y comportamiento responsive. | Permiten revisar el comportamiento de la aplicación en diferentes resoluciones, dispositivos simulados y navegadores. |
| PostgreSQL / pgAdmin | Validación de datos y pruebas de integridad. | Permiten comprobar que la información generada por las pruebas sea almacenada correctamente y que se mantengan las relaciones e integridad de los datos. |
| Git / GitHub | Control de versiones y seguimiento de incidencias. | Permite mantener el historial de cambios, gestionar correcciones y facilitar el trabajo colaborativo durante el proceso de pruebas. |

### 3.4.5 Recursos para la ejecución de las pruebas

#### Recursos humanos:

La ejecución de las pruebas requiere la participación de los integrantes del equipo de desarrollo, quienes tendrán responsabilidades relacionadas con la preparación de datos, ejecución de casos de prueba, registro de resultados, análisis de errores y corrección de incidencias. También se considera la participación de usuarios de prueba para validar aspectos relacionados con usabilidad y accesibilidad.

#### Recursos técnicos:

Se requiere un equipo de desarrollo con acceso al sistema Atlas, navegador web actualizado, entorno de ejecución para React y Express, base de datos PostgreSQL y las herramientas de pruebas indicadas anteriormente. Para las pruebas de compatibilidad se considerarán las últimas dos versiones de Chrome, Firefox, Safari y Edge, además de resoluciones correspondientes a computadores, tablets y dispositivos móviles.

#### Entorno de prueba:

Las pruebas deberán ejecutarse inicialmente en un ambiente controlado de desarrollo o QA, utilizando una base de datos de prueba y datos ficticios. Esto permite realizar pruebas funcionales, de integración, seguridad y rendimiento sin afectar información real. Una vez corregidos los defectos encontrados, se deberán repetir las pruebas correspondientes y ejecutar pruebas de regresión antes de considerar el sistema listo para su validación final.

## 4. Estrategia de pruebas y justificación

La estrategia de pruebas para la plataforma Atlas define los lineamientos metodológicos, el alcance operativo y el enfoque de mitigación de riesgos que gobernarán el proceso de aseguramiento de calidad (QA) y seguridad (DevSecOps). Esta estrategia se diseña bajo las directrices del estándar IEEE/ISO/IEC 29119 y se alinea con los requerimientos de confiabilidad, rendimiento y cumplimiento legal chileno (Ley N° 21.663 de Ciberseguridad y Ley N° 21.719 de Protección de Datos Personales).

### 4.1 Alcance de las pruebas (Scope)

Para garantizar un control eficiente de los recursos y focalizar la verificación en los componentes críticos del software, se delimita formalmente el alcance:

A. Componentes dentro del alcance (In-Scope)

- Frontend SPA (React 19.1.1): Validación de vistas, navegación por roles (Admin/Editor), responsividad multidispositivo (desktop, tablet, móvil), formularios reactivos y cumplimiento de accesibilidad WCAG 2.1 Nivel AA adaptado a personas adultas mayores (etiquetas ARIA, tipografía≥12px y contraste).
- Backend API REST (Node.js / Express 4.19.2): Verificación exhaustiva de controladores, middlewares de autorización (RBAC), mecanismos de autenticación y revocación de tokens JWT, validación estricta de payloads server-side y manejo estandarizado de excepciones (códigos HTTP y supresión de stack traces).
- Base de Datos y Persistencia (PostgreSQL): Consistencia transaccional ACID, validación de borrado lógico (soft delete) en clientes (RF-4.3), segregación multinquilino (multi-tenancy) y persistencia inmutable de pistas de auditoría (RF-6.1).
- Gestión Documental y Archivos: Subida, integridad, almacenamiento interno, referenciación en base de datos y descarga controlada de archivos en formatos autorizados (PDF, DOC, PPT, JPG, PNG) bajo la cota de 10 segundos (RF-5.2, RF-5.3).
- Seguridad y Rendimiento: Evaluación de controles contra OWASP Top 10 (especialmente IDOR/BOLA, SQLi y XSS), cifrado TLS y evaluación de latencia (<300<300ms en operaciones CRUD) bajo concurrencia objetivo de 200 usuarios.

B. Componentes fuera del alcance (Out-Scope)

- Pruebas de penetración física o ingeniería social contra el personal de CreaLab SpA o las empresas usuarias.
- Seguridad perimetral y física de los centros de datos donde se aloja la infraestructura en la nube (la responsabilidad recae en el proveedor de infraestructura IaaS/PaaS).
- Pruebas de pasarelas de pago externas o emisión electrónica tributaria (fuera del alcance funcional de la versión 1.0 de Atlas).
- Rendimiento de redes móviles de terceros en situaciones de degradación extrema o sin cobertura.

### 4.2 Enfoque metodológico y niveles de prueba

Se adopta un enfoque mixto y progresivo (Shift-Left Testing) que integra aseguramiento de calidad y pruebas de seguridad a lo largo de los siguientes niveles:

1. Pruebas Estáticas (SAST y SCA): Análisis de código fuente y escaneo automatizado de dependencias NPM para detectar tempranamente librerías vulnerables y defectos sintácticos de seguridad antes del despliegue en entornos de pruebas.

2. Pruebas de Integración y API: Ejecución automatizada mediante colecciones de Postman y pruebas de backend con Jest/Supertest, validando la interacción de la API Express con la base de datos PostgreSQL, la vigencia del ciclo de vida de los JWT y la restricción efectiva de permisos según el modelo RBAC.

3. Pruebas No Funcionales Especializadas:

    - Carga y Rendimiento: Simulación de carga concurrente (hasta 200 usuarios concurrentes) mediante Apache JMeter / k6 contra los endpoints de consulta de clientes y contratos, garantizando que el tiempo de respuesta no supere los 300 ms.

    - Seguridad Dinámica (DAST): Escaneo con OWASP ZAP en el ambiente de QA para comprobar la ausencia de inyecciones SQL, vulnerabilidades XSS y exposición involuntaria de información sensible.

4. Pruebas Manuales y de Aceptación (UAT): Sesiones guiadas de validación funcional y de usabilidad con 5 usuarios representativos (incluyendo adultos mayores), evaluando la claridad visual, flujos cognitivos simplificados y legibilidad de contratos y mensajes de confirmación.

### 4.3 Matriz de evaluación y mitigación de riesgos de prueba

La priorización de las pruebas se fundamenta en un análisis de riesgos técnicos y de negocio:

| Riesgo Identificado | Impacto | Probabilidad | Nivel de riesgo | Enfoque de Prueba / Mitigación en el Plan |
|---|---|---|---|---|
| Fuga de datos entre empresas (IDOR / Multi-tenant Leak): Un usuario manipula identificadores de la API para ver clientes o contratos de otra empresa. | Crítico | Media | Alto | Pruebas de integración de seguridad de caja negra y caja gris alterando IDs en peticiones REST autenticadas como Editor y Admin de distintas empresas. |
| Robo o reutilización de tokens JWT: Sesiones inactivas o tokens no invalidados tras el cierre de sesión (logout). | Alto | Media | Alto | Verificación de expiración estricta de tokens, listas de revocación en backend y pruebas funcionales de acceso con tokens caducados (RF-1.2, NFR-SEG-9). |
| Degradación del servicio ante concurrencia: Latencia mayor a 300 ms o caídas del sistema ante más de 200 usuarios concurrentes. | Alto | Alta | Alto | Ejecución de pruebas de estrés y carga sostenida con JMeter monitoreando el consumo de memoria en Node.js y pools de conexión de PostgreSQL. |
| Barreras de accesibilidad para adultos mayores: Dificultad para operar la interfaz por botones pequeños o contraste insuficiente. | Medio | Media | Medio | Auditoría con herramientas automatizadas de accesibilidad (Lighthouse / Axe-core) complementada con pruebas de usuario presenciales (5 participantes). |
| Pérdida accidental de información por eliminación permanente: Pérdida definitiva de clientes con contratos históricos vinculados. | Alto | Baja | Medio | Pruebas funcionales del flujo de confirmación modal y verificación en base de datos del flag de borrado lógico (soft delete) en tablas relacionales (RF-4.2). |

### 4.4 Criterios de suspensión y reanudación

- Criterios de Suspensión: El ciclo de pruebas se suspenderá inmediatamente si se detecta un defecto bloqueante (Showstopper), tal como:
    - Indisponibilidad total del entorno de pruebas o caída de la base de datos PostgreSQL.
    - Falla crítica en el endpoint de autenticación /api/auth/login que impida el acceso a los usuarios de prueba.
    - Brecha de seguridad crítica evidente (ej. omisión total de autorización RBAC o inyección SQL trivial).

- Criterios de Reanudación: Las pruebas se reanudarán una vez que el equipo de desarrollo libere una nueva versión correctiva (hotfix) en el ambiente QA, se valide la corrección del defecto bloqueante y se aplique una batería básica de pruebas de regresión (Smoke Test).

### 4.5 Justificación de la estrategia frente a normativas y estándares

Esta estrategia se fundamenta y justifica en los siguientes pilares:

1. Cumplimiento de la Ley N° 21.663 (Ciberseguridad en Chile): La priorización de pruebas sobre registros de auditoría inmutables (Audit Trails) y control de acceso responde a la exigencia de trazabilidad y reporte ante incidentes exigidos por la Agencia Nacional de Ciberseguridad (ANCI).
2. Cumplimiento de la Ley N° 21.719 (Protección de Datos Personales): El alcance cubre la protección criptográfica en tránsito (TLS) y reposo, minimización de datos y validación del borrado lógico para resguardar los derechos de los titulares de datos (RUT, cuentas bancarias e información comercial).
3. Estándar ISO/IEC 25010: Garantiza la evaluación equilibrada de características de calidad clave: Adecuación Funcional (cobertura de RF-1 a RF-6), Eficiencia de Desempeño (umbrales de tiempo de respuesta), Usabilidad (inclusión de personas adultas mayores) y Fiabilidad (mecanismos de respaldo y recuperación).

## 5. Recursos necesarios

Para ejecutar el plan de pruebas del Sistema de Gestión Atlas se requieren recursos humanos, técnicos y de entorno que permitan verificar el cumplimiento de los requisitos funcionales, no funcionales, de seguridad, rendimiento, usabilidad, accesibilidad y compatibilidad definidos en la Especificación de Requerimientos de Software (SRS).

### 5.1 Recursos humanos

Para la ejecución y supervisión de las pruebas se consideran los siguientes roles:

| Rol | Responsabilidades dentro del plan de pruebas |
|---|---|
| **Tester / QA** | Diseñar, ejecutar y documentar los casos de prueba; verificar los requisitos funcionales y no funcionales; registrar defectos y generar evidencias de las pruebas realizadas. |
| **Desarrollador Frontend** | Apoyar en la corrección y análisis de defectos relacionados con React, Material UI, Tailwind, navegación, formularios, responsividad y accesibilidad. |
| **Desarrollador Backend** | Analizar y corregir problemas relacionados con la API REST, autenticación, autorización, validación de entradas, JWT, lógica de negocio y conexión con PostgreSQL. |
| **Analista de seguridad** | Ejecutar o apoyar pruebas de seguridad sobre autenticación, autorización, gestión de sesiones, validación de entradas y vulnerabilidades como XSS, CSRF y SQL Injection. |
| **Administrador de base de datos / encargado técnico** | Apoyar la preparación, configuración y restauración de la base de datos PostgreSQL, además de verificar la persistencia e integridad de los datos durante las pruebas. |

| | |
|---|---|
| **Usuarios representativos** | Participar en las pruebas de usabilidad y accesibilidad, especialmente considerando el requisito de realizar pruebas con al menos 5 usuarios representativos, incluyendo adultos mayores. |

La participación de estos roles permite cubrir las distintas áreas evaluadas por el plan, evitando que las pruebas se concentren únicamente en la funcionalidad del sistema y permitiendo evaluar también aspectos de seguridad, rendimiento, accesibilidad y operación.

## 5.2 Recursos técnicos

Los recursos técnicos necesarios se encuentran directamente relacionados con la arquitectura y tecnologías definidas para Atlas.

| Recurso | Uso dentro de las pruebas |
|---|---|
| **Aplicación Frontend en React 19.1.1** | Permitir la ejecución de pruebas sobre los flujos de registro, autenticación, dashboard, gestión de usuarios, clientes, contratos, documentos y auditoría. |
| **Backend en Node.js con Express 4.19.2** | Ejecutar y validar los endpoints REST definidos para autenticación, usuarios, empresas, clientes, contratos, documentos, roles y auditoría. |
| **Base de datos PostgreSQL** | Verificar la creación, consulta, modificación y eliminación de información, además de validar la integridad y persistencia de los datos. |
| **Almacenamiento interno de documentos** | Comprobar la carga, almacenamiento, referencia, descarga y gestión de documentos asociados a contratos. |

| | |
|---|---|
| **Navegadores web** | Ejecutar pruebas de compatibilidad utilizando las últimas dos versiones de Chrome, Firefox, Safari y Edge, según NFR-COMPAT-1. |
| **Dispositivos móviles y tablets** | Verificar la responsividad y funcionamiento de la interfaz en distintos tamaños de pantalla, de acuerdo con NFR-COMPAT-2. |
| **Equipos de escritorio / notebooks** | Ejecutar las pruebas funcionales, de seguridad, rendimiento y compatibilidad desde un entorno de escritorio. |
| **Conexión de red con TLS/HTTPS** | Validar que las comunicaciones entre frontend y backend se realicen mediante conexiones seguras, según NFR-SEG-1. |
| **Herramienta para pruebas de API** | Permitir probar directamente los endpoints REST, incluyendo autenticación, autorización, validaciones y respuestas HTTP. |
| **Herramienta de pruebas de carga** | Permitir simular usuarios concurrentes y verificar el cumplimiento de NFR-PERF-1 y NFR-PERF-3, incluyendo el escenario objetivo de 200 usuarios concurrentes. |
| **Herramientas de pruebas de seguridad** | Permitir identificar vulnerabilidades relacionadas con autenticación, autorización, inyección, XSS, CSRF, manejo de sesiones y validación de entradas. |
| **Herramienta de gestión de incidencias** | Registrar defectos encontrados durante la ejecución de las pruebas, incluyendo su prioridad, evidencia, estado y seguimiento. |

## 5.3 Recursos de entorno

Se deberá disponer de ambientes diferenciados para evitar que la ejecución de pruebas afecte información real o interfiera con otros procesos del sistema.

**Ambiente de pruebas:** deberá contener una instancia de frontend, backend, base de datos PostgreSQL y almacenamiento de documentos configurados específicamente para la ejecución del plan de pruebas. Este ambiente deberá utilizar datos de prueba controlados y usuarios ficticios.

**Ambiente de pruebas de seguridad:** deberá permitir ejecutar pruebas de seguridad y penetración sin comprometer información productiva. Se deberán disponer cuentas con diferentes roles, principalmente administrador y editor, para comprobar los mecanismos de autenticación y autorización.

**Ambiente de pruebas de rendimiento:** deberá contar con una configuración que permita generar carga concurrente y medir los tiempos de respuesta de las operaciones. Se deberá comprobar especialmente el requisito de soportar hasta 200 usuarios concurrentes y el tiempo de respuesta inferior a 300 ms para operaciones CRUD simples bajo carga normal.

**Ambiente de pruebas de compatibilidad:** deberá considerar navegadores compatibles y dispositivos con diferentes resoluciones y tamaños de pantalla, incluyendo computadores, tablets y teléfonos móviles.

**Ambiente de pruebas de usabilidad y accesibilidad:** deberá permitir la participación de al menos 5 usuarios representativos, incluyendo adultos mayores, con el objetivo de verificar que los flujos principales sean comprensibles, accesibles y no presenten bloqueos importantes.

### 5.4 Datos de prueba

Además de los recursos humanos, técnicos y de entorno, se deberán preparar datos de prueba controlados que permitan cubrir los distintos escenarios definidos en el SRS.

Estos datos deberán considerar, entre otros:

- Usuarios con rol **administrador**.
- Usuarios con rol **editor**.
- Usuarios sin autenticación.
- Empresas de prueba.
- Clientes con información válida e inválida.
- Contratos con diferentes fechas de inicio y término para comprobar los estados **activo, vence y expirado**.
- Documentos en los formatos permitidos: **PDF, DOC, PPT, JPG y PNG**.
- Datos incorrectos o maliciosos para validar las medidas de seguridad y sanitización de entradas.
- Credenciales válidas e inválidas para comprobar autenticación y gestión de sesiones.
- Datos suficientes para ejecutar pruebas de paginación y filtros.
- Registros que permitan comprobar la generación de eventos de auditoría.

Los datos utilizados deberán ser ficticios o anonimizados, evitando utilizar información personal real durante la ejecución de las pruebas.

## 5.5 Consideraciones de configuración

La configuración de los ambientes de prueba deberá mantener correspondencia con la arquitectura especificada para Atlas: frontend desarrollado en React, backend basado en Node.js/Express y base de datos PostgreSQL.

Asimismo, se deberá verificar que los mecanismos de seguridad requeridos se encuentren habilitados durante las pruebas, incluyendo comunicación mediante TLS, autenticación basada en JWT, hashing de contraseñas mediante bcrypt, control de acceso por roles y validación de entradas.

La disponibilidad de estos recursos permitirá ejecutar el plan de pruebas de manera controlada y obtener evidencias suficientes para determinar si Atlas cumple con los requisitos funcionales, de seguridad, rendimiento, usabilidad, accesibilidad, disponibilidad y compatibilidad establecidos en el SRS.

## 6. Diseño de casos de prueba

Se presentan ocho casos de prueba representativos de los requisitos funcionales y no funcionales definidos en la Especificación de Requisitos de Software (ERS) de Atlas. Los casos permiten verificar funcionalidades relacionadas con autenticación, control de acceso, gestión de clientes, contratos, documentos, auditoría y rendimiento.

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-01** | Autenticación de usuario con credenciales válidas | RF-1.1 |
| **DESCRIPCIÓN** | | |
| Verificar que un usuario registrado pueda iniciar sesión correctamente utilizando un nombre de usuario y contraseña válidos. | | |
| **PRECONDICIONES** | | |
| ● El usuario debe encontrarse previamente registrado.<br>● El usuario debe estar activo en el sistema.<br>● El sistema debe encontrarse disponible.<br>● Debe existir una empresa asociada al usuario. | | |
| **PASOS** | | |
| 1- Acceder a la página de inicio de sesión.<br>2- Ingresar un nombre de usuario válido.<br>3- Ingresar la contraseña correspondiente.<br>4- Seleccionar la opción Iniciar sesión.<br>5- Verificar la respuesta del sistema.<br>6- Comprobar que el usuario sea redirigido al Dashboard. | | |
| **RESULTADO ESPERADO** | | |
| El sistema debe validar las credenciales correctamente, autenticar al usuario y generar un token de sesión JWT. Posteriormente, el usuario debe acceder al Dashboard correspondiente a su empresa. | | |
| **CRITERIO DE ACEPTACIÓN** | | |
| La prueba se considera exitosa si el usuario puede iniciar sesión utilizando credenciales válidas, recibe un token de sesión válido y accede correctamente al Dashboard. | | |
| **DATOS DE PRUEBA** | | |
| - Nombre de usuario: admin@empresa.cl<br>- Contraseña: Admin123 | | |
| **TIPO DE PRUEBA** | | |
| Prueba funcional – Positiva | | |

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-02** | Restricción de operaciones para usuario Editor | RF-3.2 / RB-4 |

**DESCRIPCIÓN**

Verificar que un usuario que posee el rol Editor pueda visualizar información, pero no pueda realizar operaciones de creación, edición o eliminación.

**PRECONDICIONES**

- Debe existir una empresa registrada.
- Debe existir un usuario asociado a la empresa con rol Editor.
- Deben existir clientes registrados.
- El usuario Editor debe haber iniciado sesión correctamente.

**PASOS**

1. Iniciar sesión utilizando un usuario con rol Editor.
2. Acceder al módulo **Gestión de clientes**.
3. Visualizar el listado de clientes.
4. Intentar crear un nuevo cliente.
5. Intentar modificar un cliente existente.
6. Intentar eliminar un cliente existente.
7. Verificar las respuestas del sistema.

**RESULTADO ESPERADO**

El sistema debe permitir al usuario Editor visualizar la información de los clientes, pero debe impedir las operaciones de creación, edición y eliminación.
El backend también debe rechazar las solicitudes no autorizadas realizadas directamente sobre los endpoints protegidos.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si el usuario Editor puede consultar información, pero no puede crear, modificar ni eliminar registros.

**DATOS DE PRUEBA**

- Usuario: editor@empresa.cl
- Rol: Editor
- Operaciones:
    - Crear cliente.
    - Editar cliente.
    - Eliminar cliente.

**TIPO DE PRUEBA**

Prueba funcional – Seguridad y control de acceso

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-03** | Creación de cliente con información válida | RF-4.1 / RF-4.3 |

**DESCRIPCIÓN**

Verificar que un usuario autorizado pueda crear un cliente ingresando correctamente los datos solicitados por el sistema.

**PRECONDICIONES**

- El usuario debe estar autenticado.
- El usuario debe pertenecer a una empresa.
- El usuario debe contar con permisos para crear clientes.
- El sistema debe encontrarse disponible.

**PASOS**

1. Iniciar sesión con un usuario autorizado.
2. Acceder al módulo **Gestión de clientes**.
3. Seleccionar **Crear cliente**.
4. Ingresar el nombre del cliente.
5. Ingresar la descripción.
6. Ingresar la dirección.
7. Ingresar la información de contacto.
8. Seleccionar **Guardar**.
9. Consultar el listado de clientes.

**RESULTADO ESPERADO**

El sistema debe validar los datos ingresados y crear correctamente el nuevo cliente. El registro debe quedar asociado a la empresa del usuario y aparecer en el listado correspondiente.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si el cliente es creado correctamente, sus datos quedan almacenados y posteriormente puede visualizarse en el listado de clientes.

**DATOS DE PRUEBA**

- Nombre: Empresa ABC
- Descripción: Cliente de prueba
- Dirección: Av. Providencia 123
- Contacto: contacto@abc.cl

**TIPO DE PRUEBA**

Prueba funcional – Positiva

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-04** | Eliminación lógica de cliente con confirmación | RF-4.2 |

**DESCRIPCIÓN**

Verificar que el sistema solicite confirmación al usuario antes de eliminar un cliente y que la eliminación sea realizada de manera lógica.

**PRECONDICIONES**

- El usuario debe estar autenticado.
- Debe existir al menos un cliente registrado.
- El usuario debe tener permisos para eliminar clientes.

**PASOS**

.Iniciar sesión con un usuario autorizado.

1. Acceder a **Gestión de clientes**.
2. Seleccionar un cliente existente.
3. Seleccionar la opción **Eliminar**.
4. Verificar que aparezca un mensaje de confirmación.
5. Seleccionar **Cancelar**.
6. Comprobar que el cliente continúe disponible.
7. Repetir la operación de eliminación.
8. Seleccionar **Confirmar**.
9. Consultar nuevamente el listado de clientes.

**RESULTADO ESPERADO**

Al seleccionar Cancelar, el sistema no debe realizar ninguna modificación.
Al seleccionar Confirmar, el sistema debe realizar la eliminación lógica del cliente y este no debe aparecer como registro activo.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si el sistema solicita confirmación antes de eliminar y únicamente realiza la eliminación cuando el usuario confirma la operación.

**DATOS DE PRUEBA**

- Cliente: Empresa ABC
- Acción: **Eliminar**
- Primera respuesta: **Cancelar**
- Segunda respuesta: **Confirmar**

**TIPO DE PRUEBA**

Prueba funcional – Usabilidad

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-05** | Cálculo automático del estado del contrato | RF-5.4 |

**DESCRIPCIÓN**

Verificar que el sistema determine correctamente el estado de un contrato considerando su fecha de inicio y fecha de término.

**PRECONDICIONES**

- El usuario debe estar autenticado.
- Debe existir un cliente registrado.
- Debe existir un contrato asociado al cliente.
- El usuario debe contar con permisos para consultar contratos.

**PASOS**

1. Iniciar sesión en el sistema.
2. Acceder al módulo de clientes.
3. Seleccionar un cliente.
4. Acceder a los contratos asociados.
5. Seleccionar un contrato.
6. Revisar las fechas de inicio y término.
7. Verificar el estado mostrado por el sistema.
8. Comparar el estado con las fechas configuradas.

**RESULTADO ESPERADO**

El sistema debe calcular automáticamente el estado del contrato según la fecha de inicio y fecha de término, mostrando uno de los estados establecidos: activo, vence o expirado.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si el estado mostrado corresponde al período en que se encuentra el contrato de acuerdo con sus fechas configuradas.

**DATOS DE PRUEBA**

- Cliente: Empresa ABC
- Título: Contrato de prestación de servicios
- Fecha de inicio: 01/09/2026
- Fecha de término: 30/09/2026

**TIPO DE PRUEBA**

Prueba funcional – Regla de negocio

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-06** | Carga de documento PDF asociado a contrato | RF-5.2 / RF-5.3 |

**DESCRIPCIÓN**

Verificar que un usuario pueda adjuntar un documento en formato permitido a un contrato y que el documento quede almacenado y asociado correctamente.

**PRECONDICIONES**

- El usuario debe estar autenticado.
- Debe existir un cliente registrado.
- Debe existir un contrato asociado al cliente.
- Debe existir un documento en uno de los formatos permitidos.

**PASOS**

1. Iniciar sesión.
2. Acceder al módulo de clientes.
3. Seleccionar el cliente correspondiente.
4. Seleccionar el contrato.
5. Seleccionar **Adjuntar documento**.
6. Seleccionar el archivo de prueba.
7. Confirmar la carga.
8. Esperar la respuesta del sistema.
9. Consultar los documentos asociados al contrato.
10. Intentar descargar el documento.

**RESULTADO ESPERADO**

El sistema debe aceptar el documento PDF, almacenarlo correctamente en el almacenamiento interno y registrar su referencia en la base de datos.
El documento debe quedar disponible para su posterior descarga.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si el documento es aceptado, queda asociado al contrato correcto, puede ser consultado y puede descargarse en un tiempo no superior a 10 segundos.

**DATOS DE PRUEBA**

- Contrato: Contrato de prestación de servicios
- Archivo: contrato-prueba.pdf
- Formato: PDF

**TIPO DE PRUEBA**

Prueba funcional – Integración

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-07** | Registro de creación de cliente en auditoría | RF-6.1 |

**DESCRIPCIÓN**

Verificar que el sistema registre automáticamente en el módulo de auditoría una acción crítica realizada por un usuario.

**PRECONDICIONES**

- El usuario debe estar autenticado.
- El usuario debe tener permisos para crear clientes.
- El módulo de auditoría debe encontrarse disponible.

**PASOS**

1. Iniciar sesión con un usuario autorizado.
2. Acceder al módulo de clientes.
3. Seleccionar **Crear cliente**.
4. Ingresar información válida.
5. Guardar el nuevo cliente.
6. Acceder al módulo **Auditoría**.
7. Buscar las acciones realizadas por el usuario.
8. Revisar el registro correspondiente a la creación del cliente.

**RESULTADO ESPERADO**

El sistema debe generar automáticamente un registro de auditoría correspondiente a la creación del cliente. El registro debe permitir identificar al usuario, la acción realizada y el momento en que se ejecutó.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si después de realizar una acción crítica se genera un registro de auditoría que permita identificar al usuario, la acción realizada, la fecha y hora y el recurso afectado.

**DATOS DE PRUEBA**

- Usuario: admin@empresa.cl
- Acción: Creación de cliente
- Cliente: Empresa ABC

**TIPO DE PRUEBA**

Prueba funcional – Auditoría

| ID | NOMBRE | REQUERIMIENTO |
|---|---|---|
| **CP-08** | Tiempo de respuesta de consulta de clientes | NFR-PERF-1 |

**DESCRIPCIÓN**

Verificar que una operación CRUD simple presente un tiempo de respuesta inferior a 300 milisegundos bajo condiciones normales de carga.

**PRECONDICIONES**

- El sistema debe encontrarse disponible.
- El backend debe estar correctamente configurado.
- La base de datos PostgreSQL debe encontrarse disponible.
- Deben existir datos suficientes para realizar la consulta.
- La prueba debe ejecutarse bajo condiciones normales de carga.

**PASOS**

1. Iniciar el ambiente de pruebas.
2. Preparar los datos necesarios en la base de datos.
3. Ejecutar una solicitud de consulta de clientes.
4. Medir el tiempo transcurrido entre la solicitud y la respuesta.
5. Repetir la prueba varias veces.
6. Registrar los tiempos obtenidos.
7. Comparar los resultados con el límite establecido en el ERS.

**RESULTADO ESPERADO**

La operación debe responder correctamente y presentar un tiempo de respuesta inferior a 300 ms bajo condiciones normales de carga.

**CRITERIO DE ACEPTACIÓN**

La prueba se considera exitosa si la operación CRUD evaluada mantiene un tiempo de respuesta inferior a 300 ms, sin errores durante la ejecución.

**DATOS DE PRUEBA**

Endpoint:

GET /api/empresas/{id}/clients

Condición:

- Carga normal.
- Solicitud de consulta de clientes.
- Medición del tiempo de respuesta en milisegundos.

**TIPO DE PRUEBA**

Prueba no funcional – Rendimiento

| ID | Caso de prueba | Requerimiento | Tipo |
|---|---|---|---|
| **CP-01** | Autenticación de usuario | RF-1.1 | Funcional |
| **CP-02** | Restricción de permisos del Editor | RF-3.2 / RB-4 | Funcional / Seguridad |
| **CP-03** | Creación de cliente | RF-4.1 / RF-4.3 | Funcional |
| **CP-04** | Eliminación lógica de cliente | RF-4.2 | Funcional / Usabilidad |
| **CP-05** | Cálculo del estado del contrato | RF-5.4 | Funcional / Regla de negocio |
| **CP-06** | Carga de documento | RF-5.2 / RF-5.3 | Funcional / Integración |
| **CP-07** | Registro de auditoría | RF-6.1 | Funcional / Auditoría |
| **CP-08** | Tiempo de respuesta CRUD | NFR-PERF-1 | No funcional / Rendimiento |